

```

require('dotenv').config();

const path = require('path');
const express = require('express');
const cors = require('cors');
const Database = require('better-sqlite3');
const { nanoid } = require('nanoid');

const PORT = process.env.PORT || 3000;
const GROQ_API_KEY = process.env.GROQ_API_KEY;
const GROQ_MODEL = process.env.GROQ_MODEL || 'llama-3.3-70b-versatile';

if (!GROQ_API_KEY) {
  console.warn('! No GROQ_API_KEY set. Add it to your .env file (locally) or yc
}

// ----- Database -----
const db = new Database(path.join(__dirname, 'chat.db'));
db.pragma('journal_mode = WAL');

db.exec(`
  CREATE TABLE IF NOT EXISTS sessions (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL DEFAULT 'New chat',
    created_at INTEGER NOT NULL
  );

  CREATE TABLE IF NOT EXISTS messages (
    id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL,
    role TEXT NOT NULL,
    content TEXT NOT NULL,
    created_at INTEGER NOT NULL,
    FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
  );
`);

const stmts = {
  listSessions: db.prepare('SELECT id, title FROM sessions ORDER BY created_at DE
  getSession: db.prepare('SELECT id, title FROM sessions WHERE id = ?'),
  insertSession: db.prepare('INSERT INTO sessions (id, title, created_at) VALUES
  renameSession: db.prepare('UPDATE sessions SET title = ? WHERE id = ?'),
  deleteSession: db.prepare('DELETE FROM sessions WHERE id = ?'),
  deleteMessagesForSession: db.prepare('DELETE FROM messages WHERE session_id = ?

```

```

    listMessages: db.prepare('SELECT role, content FROM messages WHERE session_id =
    insertMessage: db.prepare('INSERT INTO messages (id, session_id, role, content,
};

// ----- App -----
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// Health check
app.get('/api/health', (req, res) => {
  res.json({ ok: true, model: GROQ_MODEL });
});

// List sessions
app.get('/api/sessions', (req, res) => {
  res.json(stmts.listSessions.all());
});

// Create session
app.post('/api/sessions', (req, res) => {
  const id = nanoid();
  const title = 'New chat';
  stmts.insertSession.run(id, title, Date.now());
  res.status(201).json({ id, title });
});

// Delete session
app.delete('/api/sessions/:id', (req, res) => {
  const { id } = req.params;
  stmts.deleteMessagesForSession.run(id);
  stmts.deleteSession.run(id);
  res.status(204).end();
});

// List messages for a session
app.get('/api/sessions/:id/messages', (req, res) => {
  const { id } = req.params;
  const session = stmts.getSession.get(id);
  if (!session) return res.status(404).json({ error: 'Session not found' });
  res.json(stmts.listMessages.all(id));
});

// Send a message, get an AI reply
app.post('/api/sessions/:id/messages', async (req, res) => {
  const { id } = req.params;

```

```

const { message } = req.body || {};

if (!message || typeof message !== 'string' || !message.trim()) {
  return res.status(400).json({ error: 'Message text is required.' });
}

const session = stmts.getSession.get(id);
if (!session) return res.status(404).json({ error: 'Session not found' });

if (!GROQ_API_KEY) {
  return res.status(500).json({ error: 'No GROQ_API_KEY set on the server.' });
}

const now = Date.now();
stmts.insertMessage.run(nanoid(), id, 'user', message, now);

// Auto-title the session from the first user message
if (session.title === 'New chat') {
  const title = message.trim().slice(0, 48) + (message.trim().length > 48 ? '...'
  stmts.renameSession.run(title, id);
}

try {
  const history = stmts.listMessages.all(id).map((m) => ({ role: m.role, conten
  const reply = await callGroq(history);

  stmts.insertMessage.run(nanoid(), id, 'assistant', reply, Date.now());
  res.json({ reply });
} catch (err) {
  console.error('Groq API error:', err.message);
  res.status(502).json({ error: 'The AI service failed to respond. Please try a
}
});

// ----- Groq call -----
async function callGroq(history) {
  const response = await fetch('https://api.groq.com/openai/v1/chat/completions',
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${GROQ_API_KEY}`,
    },
    body: JSON.stringify({
      model: GROQ_MODEL,
      messages: [
        {
          role: 'system',

```

```

        content: 'You are Ember, a warm, sharp, and concise AI assistant. Keep
    },
    ...history,
  ],
  temperature: 0.7,
  })),
});

if (!response.ok) {
  const text = await response.text().catch(() => '');
  throw new Error(`Groq API responded with ${response.status}: ${text}`);
}

const data = await response.json();
return data.choices?.[0]?.message?.content?.trim() || '(no response)';
}

// Fallback: serve the SPA for any other route
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

app.listen(PORT, () => {
  console.log(`🔥 Ember is running at http://localhost:${PORT}`);
});

```